Integrated Surveillance and Access Control System (IASS)

Development Progress Report

Prepared for project review -- summary of software, firmware, and hardware progress to date, current integration status, verification results, and remaining work.

Software architecture version	V2 -- layered
Firmware version	V3.0.0
Automated software tests passing	32 / 32
Firmware build status	Clean build, zero warnings (-Wall -Wextra -Wshadow -Wconversion)
Firmware flash / RAM usage	22.9% / 35.2% of ATmega328P capacity

1. Executive Summary

IASS is a layered surveillance and access-control system pairing a host application (camera ingestion, AI-based person detection/recognition, security decisioning, and hardware orchestration) with dedicated ATmega328P actuator firmware (camera-mount servos, door locks, and alarm outputs). This report summarizes what has been built, what has been independently verified rather than assumed, and what remains before field deployment.

The software and firmware are the most mature parts of the project and are covered in the most detail below. The mechanical/electrical design of the central controller enclosure is in progress; renders were referenced for this report but were not available in the current working session, so that section is intentionally left as a placeholder pending those files -- see Section 4.

Progress by area

Software -- core layers	90%	All 8 layers implemented, tested, and independently verified end-to-end.
Software -- AI perception	85%	Real neural detection + recognition models integrated and benchmarked.
Software -- scenarios	80%	Five scenario types implemented and tested; user-configurable rule builder not yet built.
Software -- configuration UI	25%	Dashboard/streaming/personnel UI complete; topology & scenario-builder UI designed, not built.
Firmware	85%	V3 architecture complete, compiles clean, host-compatible; physical hardware-in-loop testing pending.
Host <-> firmware integration	90%	Protocol formally verified byte-for-byte; one real gap found and fixed this cycle.
Hardware (mechanical/electrical)	15%	Central controller schematic/enclosure in progress; renders pending inclusion.

2. Software Progress

The host software was restructured from an earlier, tangled prototype into eight independently testable layers, each exposing a single documented contract to the layer above and below it. This mirrors how the project's own scenario flow is meant to read end to end: a camera sees someone, the AI identifies them, the security layer decides whether that's authorized, the hardware-control layer works out which physical door/alarm that decision applies to, and the hardware-interface layer is the only code that actually talks to the wire.

Layer	Responsibility
UI	Single-page dashboard (security, personnel, hardware, logs, telemetry, settings) + camera registration view
camera_stream	Multi-source ingestion: USB/OpenCV, synthetic test source, phone-over-hotspot browser push. One capture manager + frame hub per camera.
vision_core	Shared OpenCV primitives (JPEG codec, bbox geometry) used by camera_stream and ai_perception -- no duplicated image-processing code.
ai_perception	Detection (YOLOv11 ONNX) -> quality gate -> tracking -> embedding (ArcFace) -> identity match -> cross-camera handoff.
decision_security	Authorization, dwell-time risk escalation, tailgating, repeat-visitor detection. The only layer allowed to decide who is authorized.
hardware_control	Zone/topology-based device resolution -- turns a security decision into "which physical device" ("Camera 3 covers Door 3").
hardware_interface	Wire protocol, serial transport, command transactions, safety policy (e-stop, comms-loss gate), audit trail.
hardware_state	Live "Door 3 = UNLOCKED"-style state store, updated from real device telemetry.

2.1 Camera streaming -- multi-camera capability

The camera layer supports concurrent multi-camera operation: each configured camera gets its own capture thread and its own AI worker, rather than a single global feed. This was specifically re-verified this cycle by instrumenting the AI pipeline and running the full application, pushing live frames into one camera over its network feed while a second camera's independent capture loop ran in the background:

```
AI process() calls per camera (concurrent): {'CAM-02': 3, 'CAM-01': 1}
```

Both cameras processed independently and concurrently with zero errors. Supported ingestion sources: USB/OpenCV device, a synthetic test source (no physical camera required, useful for bench testing), and a phone acting as a camera over the local network/hotspot. Each camera streams to the UI as MJPEG independently of whether AI is enabled on it.

2.2 AI perception -- detection and recognition

The AI pipeline was upgraded from lightweight classical computer-vision detectors to real neural network models, benchmarked and verified running rather than only integrated in theory:

Component	Status	Notes
Face detection	**DONE**	YOLOv11-face (ONNX), trained on WIDERFace. Replaces a Haar-cascade detector. Automatic fallback if the model file or ONNX Runtime is unavailable.
Person detection	**DONE**	YOLOv11n (COCO, person class), ONNX. Replaces an HOG detector.
Face recognition / identity	**DONE**	ArcFace (ResNet-50, WebFace600K), 512-dimension embeddings -- the same recognition architecture used in InsightFace's production pipeline. Replaces a handcrafted pixel/gradient descriptor.
Multi-object tracking	**DONE**	Per-camera track assignment with IoU-based association and configurable loss tolerance.
Cross-camera handoff	**DONE**	Recognized identities are correlated across cameras so a handoff between zones is a single tracked event, not two independent detections.
Face quality gating	**DONE**	Blur, brightness, and minimum-size checks run before a detection is trusted, reducing false matches from poor-quality frames.

Known, documented limitation: recognition currently uses un-aligned detector crops rather than 5-point-landmark-aligned faces. This still performs well above the previous handcrafted descriptor, but landmark alignment (via a paired detector such as RetinaFace, whose weights were already downloaded during evaluation) is the next accuracy lever if higher recognition precision is required.

2.3 Decision & security scenarios

The scenario engine turns perception events into hardware intents. Implemented and covered by automated tests:

- **Authorized / unauthorized access** -- door unlock on confirmed match; lock-and-optionally-alarm on denial.
- **Dwell-time risk escalation** -- an unresolved presence escalates OBSERVATION → SUSPICIOUS → HIGH_RISK → SECURITY_ALERT, each with its own configurable hardware response and cooldown.
- **Tailgating detection** -- an authorized and unauthorized person in frame together raises a critical alert.
- **Cross-camera handoff** -- a recognized identity moving between cameras keeps camera tracking engaged without re-triggering initial-detection logic.
- **Repeat-visitor detection** -- a configurable counter (default: 3 separate appearances) per identity/camera pair triggers a distinct, low-severity hardware flag (e.g. a strobe indicator) separate from the security-escalation path, since a repeat visitor is not necessarily a threat.

Not yet built: a UI for defining *new* trigger → action rules without editing configuration directly (e.g., "if X happens N times, do Y"). The underlying rule *evaluation* exists for the five scenarios above; a general-purpose, user-configurable rule builder is designed but not implemented -- see Section 5.

2.4 Hardware control & interface

The hardware-control layer resolves a security decision to a specific physical device via a zone/topology model (which doors, alarms, and camera mounts belong to which area). The hardware-interface layer owns the actual serial protocol, independent command-safety checks (described in Section 3.3), transaction deduplication, and an audit trail of every dispatched command and its outcome.

2.5 Testing and verification

Component	Status	Notes
Automated test suite	**PASS**	32 / 32 tests passing, covering hardware scenarios, the wire protocol contract, the AI pipeline, and safety/e-stop wiring.
Application boot	**PASS**	Full application starts cleanly with all 54 API routes registered.
Packaging	**DONE**	Proper Python packaging added; the test suite and application run without manual environment configuration.
Hardware bench-test tool	**DONE**	Standalone script exercises the firmware directly over serial (heartbeat, servo sweep, door lock/unlock, siren/strobe) without requiring the full application or a camera.

3. Firmware Progress

The ATmega328P firmware controls camera-mount servos, door lock actuators, and alarm outputs. This cycle's work focused on adding a formal command-handling layer and a persistent safety latch, while preserving full compatibility with the existing communication protocol -- verified by compiling and re-tracing every command's exact byte encoding, not by inspection alone.

3.1 Layered architecture

Layer	Responsibility
drivers/	GPIO, UART (interrupt-driven ring buffer), hardware timers, interrupt vectors.
protocol/	Frame parsing and CRC16 validation for both supported wire formats.
actuators/	Servo (software-PWM, ramped movement), door lock (auto-relock timer), and alarm (siren/strobe, timed or persistent) state machines, each driven by a 1 ms scheduler tick.
command/ (new)	Command router (decodes either wire format into one normalized structure) → dispatch table → one handler function per action. Replaces logic that was previously embedded directly in the main program loop.
system/	State storage plus an independent safety module: watchdog, heartbeat/communication-loss failsafe, and the new persistent emergency-stop latch.

3.2 Communication protocol

The firmware supports two wire formats simultaneously so the current host works unmodified while the protocol can grow: a fixed 7-byte legacy frame (checksum-protected) used by the current host application, and a versioned, CRC16-protected protocol carrying transaction IDs, correlation IDs, and command expiry (TTL) for future use. Every legacy command's exact bit encoding was re-traced against the original implementation during this cycle's restructuring to guarantee no behavior changed for the current host.

3.3 Safety systems

Component	Status	Notes
Watchdog	**DONE**	~2 second hardware watchdog; a stalled control loop resets the MCU.
Communication-loss failsafe	**DONE**	No heartbeat for 5 seconds: both doors lock, camera mounts return to home position, siren silences, and a visual strobe indicator engages -- so the loss is visible on site even with no host to report it.

Component	Status	Notes
Persistent emergency-stop latch	**NEW / DONE**	Emergency-stop now blocks door-unlock and alarm-trigger commands until an explicit, authorized clear command is received, rather than being a one-shot action. Enforced independently on the firmware -- the host's own equivalent safety check exists to fail fast, not as the only line of defense.
Command replay/staleness protection	**DONE**	Expired or duplicate rich-protocol commands are rejected before any actuator is touched.

3.4 Build verification

The firmware was compiled with a real AVR toolchain under strict warning flags (`-Wall -Wextra -Wshadow -Wconversion`) with zero warnings or errors. Resulting resource usage on the ATmega328P target:

Resource	Used	Available	Utilization
Program (flash)	7,512 bytes	32,768 bytes	22.9%
Data (RAM)	720 bytes	2,048 bytes	35.2%

Comfortable headroom remains on both flash and RAM for future actuator types or protocol extensions.

3.5 Actuator coverage

- **Camera-mount servos** (x3) -- smooth, speed-ramped movement via software PWM; independently addressable.
- **Door lock actuators** (x2) -- commanded lock/unlock with automatic re-lock timeout; sensor input supported in the design, not yet enabled on the current prototype wiring.
- **Alarm outputs** (x2) -- siren and strobe channels, independently triggerable, either timed or persistent-until-cleared.

3.6 Host ↔ firmware compatibility (verified this cycle)

The host and firmware were checked against each other directly rather than assumed compatible because they were each developed correctly in isolation. One real integration gap was found and fixed: the host had no way to issue the firmware's new emergency-stop-clear command, so the two sides' safety states could not be kept in sync. This is now wired end to end and covered by an automated test asserting the command actually reaches the hardware driver with the correct parameters.

4. Hardware Progress

Note on this section: the central-controller schematic and enclosure renders referenced for this report were not available in the current working session and could not be included. This section reflects what is documented in the firmware's own hardware register map and safety notes; the mechanical/electrical design itself should be attached separately once the render files are provided.

4.1 Current prototype device inventory

Component	Status	Notes
Camera mount servos	**DONE**	3 channels, pin-mapped and under firmware control.
Door lock actuators	**DONE**	2 channels, pin-mapped and under firmware control. Physical driver circuitry (relay/MOSFET isolation for real solenoids/strikes) required before connecting real hardware -- GPIO must not drive these loads directly.
Alarm outputs (siren/strobe)	**DONE**	2 independent channels, pin-mapped and under firmware control.
Door position sensors	**DESIGNED, NOT ENABLED**	Supported in firmware; disabled on the current prototype because the assigned pins are occupied by the alarm outputs.
Central controller schematic (SolidWorks)	**PENDING**	Referenced but not available in this session -- to be inserted once provided.
Central controller enclosure	**PENDING**	Referenced but not available in this session -- to be inserted once provided.

4.2 Central controller (renders pending)

SolidWorks renders / schematic for the central controller enclosure to be inserted here once the image files are provided.

Figure 1 -- placeholder for central controller schematic/render.

5. Remaining Work

Prioritized items to reach a field-deployable system:

- **Physical hardware-in-loop bench testing** -- the bench-test tool is ready; running it against real actuators, a real buzzer, and a real LED has not yet been done in this cycle.
- **Hardware topology configuration UI** -- assigning cameras, doors, and alarms to rooms/zones through the dashboard instead of hand-edited configuration. The underlying data model already supports this; only the UI and one ID-space consolidation (video cameras vs. hardware camera mounts currently use separate identifiers) remain.
- **Scenario/rule builder UI** -- a user-configurable trigger → action rule system (e.g., "unauthorized person detected 3 times consecutively → trigger alarm") generalizing the fixed scenarios already implemented. Designed; not yet built.
- **Driver circuitry for real actuators** -- relay/MOSFET isolation, flyback protection, and correctly rated fusing before any real solenoid, strike, or siren is connected to the controller.
- **Recognition alignment upgrade** -- landmark-based face alignment ahead of the recognition network, for improved accuracy in non-frontal poses.
- **Central controller mechanical/electrical finalization** -- pending review of the SolidWorks design once available.

6. Summary

The software and firmware are both in a genuinely working, independently verified state: real neural detection and recognition models are integrated and benchmarked, the security scenario engine covers the core access-control and escalation cases with automated test coverage, and the firmware's new command layer and safety latch were built without breaking compatibility with the existing host protocol -- confirmed by direct comparison, not assumption. The two largest remaining pieces of work are user-facing configuration (topology and scenario-rule UI) and physical hardware-in-loop validation; the mechanical design of the central controller is the least mature part of the project as of this report and needs its own dedicated review once the design files are available.